

ユニット 2・第2週

システムコールと低水準入出力

📅 第2週・日程は未定 📄 演習 TD1（解答 + 生成AIチャレンジ） 🎬 アニメーション 4 本

マニュアル M2・ユニットの理論

システムコールと低水準入出力

ユニット 2 の基本の読み物（約 1.5 時間）。各節の最後には、それを示す演習問題やアニメーションへの[実践](#) → リンクがあります。数値はコースのマシン（Linux 7.0、x86-64、glibc 2.43）で TD1 のプログラムを実際に実行して得たものです。

目次

§1	ユーザーモードとカーネルモード	§4	<code>read</code> と <code>write</code> による低水準入出力
§2	システムコールの解剖	§5	呼び出しのコスト
§3	観察する： <code>man</code> 、 <code>strace</code> 、 <code>objdump</code>	§6	その先へ： <code>vDSO</code> 、 <code>io_uring</code> 、他のアーキテクチャ

§1 ユーザーモードとカーネルモード

プロセッサには少なくとも 2 つの**特権レベル**があります。x86-64 ではリングと呼ばれ、カーネルはリング 0（CPL 0）、プログラムはリング 3（CPL 3）で動きます。ユーザーモードでは、ハードウェアに触れる命令（入出力、ページテーブル、割り込みの無効化）は禁止されています。プログラムがそれを試みるとプロセッサが例外を起こし、カーネルがそのプログラムを終了させます。

この境界が他のすべての土台です。これがあるおかげで、プログラムは他のプログラムのメモリを読んだり、権限をすり抜けてディスクに書いたり、CPU を独占したりできません。カーネルにしかできないことが必要になると、プログラムは**それを頼みます**。それがシステムコールです。

§2 システムコールの解剖

プログラムが `write(1, "hola\n", 5)` と書くと、実際には Linux x86-64 の規約に従う glibc の関数（ラッパー）を呼んでいます：

1. **システムコール番号**をレジスタ `rax` に（`write` は 1、`read` は 0、`getppid` は 110）、引数を `rdi`、`rsi`、`rdx`、`r10`、`r8`、`r9` に入れます。データそのものはレジスタに入りません：`rsi` にはテキストのアドレスが入ります。
2. `syscall` 命令を実行します：CPU はカーネルモードに切り替わり、カーネルが起動時に設定した固定アドレス（`entry_SYSCALL_64`）へジャンプします。ジャンプ先をプログラムが選ぶことはできません。
3. カーネルは `rax` を `sys_call_table` の添字として使い、ハンドラ（`ksys_write`）を実行します。ハンドラはディスクリプタを検証し、プログラムのメモリからデータをコピーし（`copy_from_user`）、デバイスドライバを呼びます。

4. 結果は `rax` で戻り、`sysret` 命令が CPU をユーザーモードに戻します。
5. `rax` が -4095 から -1 の間なら、ラッパーはエラーとして扱います：コードを `errno` に入れ、-1 を返します。だからコース全体を通じて `exit_if(rc == -1, "...")` というパターンを使い、`perror` でエラーを表示するのです。

実際の出力

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -A4 -E '^[0-9a-f]+ <getppid@>'
00000000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

実践 → アニメーション A2.1・システムコールの旅

§3 観察する： `man`、`strace`、`objdump`

ソースコードがなくてもどんなプログラムでも調べられる 3 つの道具：

- `man 2 write` はシステムコールを、`man 3 printf` はライブラリ関数を、`man 3p` は POSIX 標準を説明します。
 - `strace ./program` はすべてのシステムコールを引数と結果付きで表示します。`-f` は子プロセスも追い、`-r` は呼び出し間の時間を加え、`-c` は集計を出します。
 - `objdump -d` は逆アセンブルします。libc の中の `syscall` 命令もこれで見られます。
- `strace` を使うと驚くことがあります：TD1 の `echo` は `main` に着く前に約 30 回の呼び出しを行います。これは動的ローダのもので、`libc.so.6` を開き、`mmap` でメモリにマップし、`mprotect` でそのページを保護しています。

実践 → アニメーション A2.2・`strace` をライブで

§4 `read` と `write` による低水準入出力

どのプロセスも 3 つのファイルディスクリプタを開いた状態で生まれます：0（標準入力）、1（標準出力）、2（標準エラー）。それが何を指すかはシェルが決めます：`< file`、`> file`、`2>&1`、あるいはパイプ。プログラムはそれを知らず、番号を使うだけです。

呼び出し	戻り値	注意点
<code>read(fd, buf, n)</code>	読んだバイト数 (1...n)、ファイル終端で 0、エラーで -1	n より少ないことがある (パイプ、端末、ネットワーク、シグナル)
<code>write(fd, buf, n)</code>	書いたバイト数 (1...n)、エラーで -1	部分的な書き込み (パイプやソケットが満杯)。シグナルで中断されると <code>EINTR</code>

だからコースのプログラムは繰り返します：`read_exact` は要求が満たされるまで読み、`write_all` はすべて出し切るまで書きます。また `read` と `write` がコピーするのはテキストではなくバイトです：`long` を書けばその 8 バイトがマシンの順序 (x86-64 ではリトルエンディアン) でコピーされ、構造体を書けば境界合わせのパディングもコピーされます (`struct nopad` なら 24 バイト)。

実際の出力

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
0000016
```

⚠ 典型的なバグ：残りの分だけポインタを進める

元のスライド (B25、スライド 51) の読み込み例は

`length = length - byte; buf = buf + length;` としていました。通常のファイルなら動きます (1 回目の読み込みで全部届き、`length` が 0 になる) が、パイプやソケットでは穴が空き、**バッファの外**に書き込みます。正しくは `buf += byte` とし、`EINTR` なら再試行し、0 なら止めます。A2.4 で 1 ステップずつ確かめ、`cat file | ./read_struct` で試してみましょう。

実践 → TD1、演習問題 3〜7

§5 呼び出しのコスト

カーネルへの往復には毎回固定のコストがかかります：モードの切り替え、レジスタの保存と復元、引数の検証。TD1 の `bench_read` は、1 バイトから 1 MiB までのバッファで 8 MiB を `/dev/null` にコピーしてこれを測ります。コースのマシンでの実際の結果です：

実際の出力

```
$ ./bench_read      # 8 MiB file, buffers of 1 B to 1 MiB
buffer_bytes,read_calls,seconds,MB_per_s,ns_per_iteration
1,8388609,6.4984,1.3,775
4,2097153,1.5997,5.2,763
16,524289,0.3952,21.2,754
64,131073,0.0998,84.1,761
256,32769,0.0260,322.6,794
1024,8193,0.0075,1121.4,913
4096,2049,0.0025,3351.2,1222
16384,513,0.0010,8319.5,1966
65536,129,0.0008,10834.7,6002
262144,33,0.0007,11832.2,21484
1048576,9,0.0008,10287.9,90598
```

1回の呼び出しで1バイトなら840万回の反復でコピーに数秒かかります。64 KiB なら129回で1ミリ秒未満です。モデル $\text{時間} = \text{呼び出し回数} \times c_0 + \text{バイト数} \times c_1$ を測定値に当てはめると、**1回あたり約 0.7 μs** (read + write) の通行料と、コピー1バイトあたり0.1 ns 未満のコストが得られます。実用上の結論：数 KiB 以上のバッファを使い (C ライブラリは `printf / fwrite` でそれを代わりにやってくれます、ユニット 4)、ループで1バイトずつ読むことは決してしないこと。

実践 → アニメーション A2.3・システムコールのコスト

§6 その先へ：vDSO、io_uring、他のアーキテクチャ

- **vDSO**。 `clock_gettime` のようにとても頻繁な「呼び出し」の一部はカーネルに入りません：Linux は各プロセスに小さなライブラリ (vDSO) をマップし、共有ページから時刻を読みます。だから `strace` には現れません。
- **io_uring** (Linux 5.1、2019 年)。入出力の多いプログラム向け：プログラムとカーネルがメモリ上の2つのキューを共有し、1回の呼び出しで、あるいは呼び出しなしで数百の操作を投入できます。大きなバッファと同じ考え方を呼び出しに適用したものです。
- **他のアーキテクチャ**。64 ビット ARM では命令は `svc #0` で番号は `x8`、RISC-V では `ecall` で番号は `a7` に入ります。番号はアーキテクチャごとに違いますが、考え方は同じです。
- **eBPF による可観測性**。 `bpftrace` のようなツールは、呼び出しのたびにプロセスを止める `strace` よりはるかに低いコストで、システム全体の呼び出しを数えます。